

搭配使用 Secret Manager 與 Python

來源：<https://codelabs.developers.google.com/codelabs/secret-manager-python?hl=zh-tw>

步驟數：11

本教學課程說明如何搭配 Python 使用 Secret Manager

目錄

1. 總覽
2. 設定和需求
3. 啟用 Secret Manager API
4. 安裝 Python 適用的 Secret Manager 用戶端程式庫
5. 啟動互動式 Python
6. 建立密鑰
7. 存取密鑰
8. 搭配使用 Secret Manager 與 Cloud Functions
9. 編寫 Cloud 函式以存取 Secret
10. 部署 Cloud Function
11. 恭喜！

步驟 1 · 約 1 分鐘

1. 總覽

在本程式碼研究室中，您將著重於在 Python 中使用 Secret Manager。

Secret Manager 可讓您以二進位 blob 或文字字串的形式儲存、管理及存取密鑰。只要具備適當權限，就能查看密鑰內容。

Secret Manager 適用於儲存設定資訊，例如資料庫密碼、API 金鑰或應用程式在執行階段所需的 TLS 憑證。

課程內容

- 如何使用 Cloud Shell
- 如何安裝 Python 適用的 Secret Manager 用戶端程式庫
- 如何使用 Python 用戶端程式庫建立及存取密鑰
- 如何使用 Python 用戶端程式庫存取 Cloud Functions 中的密鑰

軟硬體需求

- Google Cloud 專案
- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 熟悉 Python 3 的使用方式

問卷調查

步驟 2 · 約 2 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button, which is circled in blue. Below this is a 'Select a project' section with a search bar. Further down is the 'New Project' section. It contains a 'Project name' field with the text 'my-kubernetes-codelab' and a question mark icon, which is also circled in blue. Below the name field is the 'Project ID' field, which contains the text 'my-kubernetes-codelab. It cannot be changed later.' and an 'EDIT' link. Below the ID field is the 'Location' field, which is set to 'No organization' and has a 'BROWSE' link. At the bottom of the 'New Project' section are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中不得重複，且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中不得重複，選定後即為專屬 ID，其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

啟用 Cloud Shell

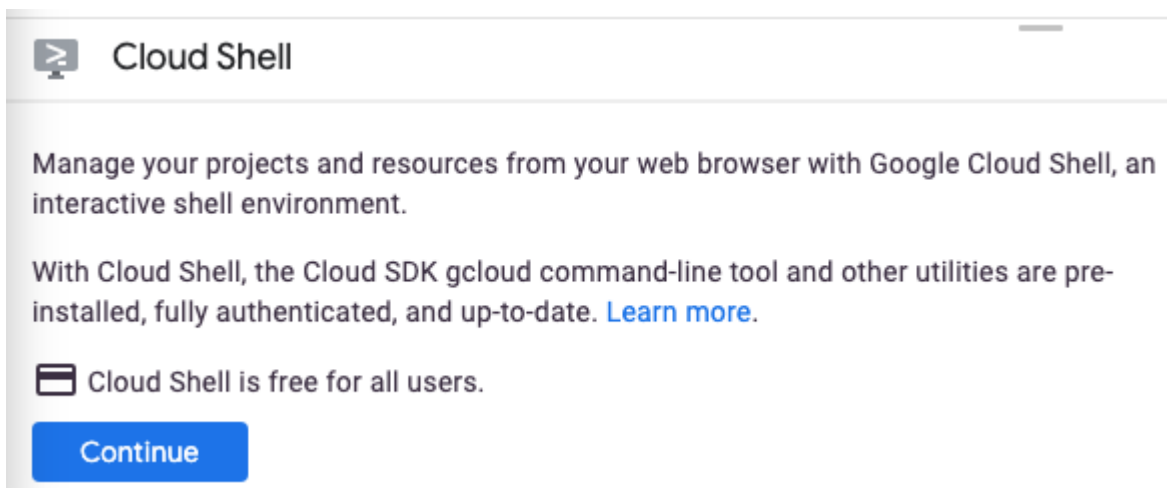
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



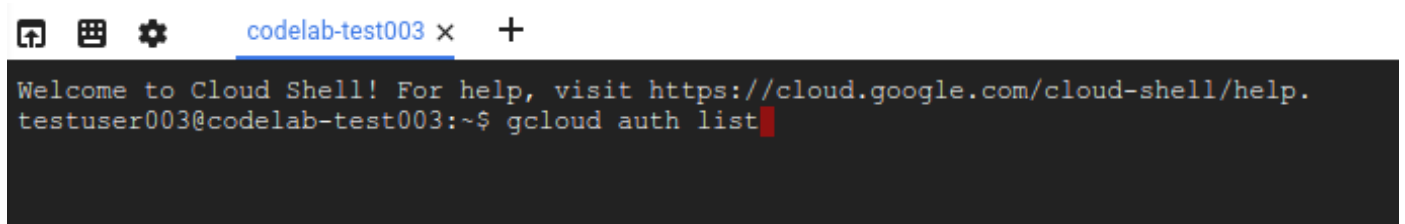
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (位於需捲動位置)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器搭載您需要的所有開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]  
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

步驟 3 · 約 1 分鐘

3. 啟用 Secret Manager API

您必須先啟用 API，才能開始使用 Secret Manager API。您可以使用 Cloud Shell 執行下列指令，啟用 API：

```
gcloud services enable secretmanager.googleapis.com
```

輸出結果應類似以下內容：

```
Operation "operations/acf.cc11852d-40af-47ad-9d59-477a12847c9e" finished  
successfully.
```

注意：如果這項操作失敗，請返回上一步並檢查設定。

步驟 4 · 約 1 分鐘

4. 安裝 Python 適用的 Secret Manager 用戶端程式庫

安裝 [Secret Manager 用戶端程式庫](#)：

```
pip3 install --user google-cloud-secret-manager==2.10.0
```

步驟 5 · 約 1 分鐘

5. 啟動互動式 Python

在本教學課程中，您會使用名為 **IPython** 的互動式 Python 解譯器，這個解譯器已預先安裝在 Cloud Shell 中。在 Cloud Shell 中執行 `ipython`，啟動工作階段：

```
ipython
```

畫面應如下所示：

```
Python 3.9.2 (default, Feb 28 2021, 17:03:44)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.3.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]:
```

注意：如有需要，您可以使用 `exit` 指令結束 IPython 工作階段。

步驟 6 · 約 3 分鐘

6. 建立密鑰

密鑰包含一或多個密鑰版本。您可以使用 `gcloud` 指令列建立，也可以使用 Python 建立。

如要使用密鑰，您必須先建立密鑰，並提供密鑰的名稱，然後新增密鑰版本，也就是密鑰的 *值*。

在 IPython 中設定專案 ID：

```
PROJECT_ID = "<PROJECT_ID>"
```

建立密鑰

將下列程式碼複製到 IPython 工作階段：

```
from google.cloud import secretmanager

def create_secret(secret_id):
    # Create the Secret Manager client.
    client = secretmanager.SecretManagerServiceClient()

    # Build the resource name of the parent project.
    parent = f"projects/{PROJECT_ID}"

    # Build a dict of settings for the secret
    secret = {'replication': {'automatic': {}}}

    # Create the secret
    response = client.create_secret(secret_id=secret_id, parent=parent, secret=secret)

    # Print the new secret name.
    print(f'Created secret: {response.name}')
```

呼叫函式來建立名為 `my_secret_value` 的新密鑰：

```
create_secret("my_secret_value")
```

您應該會看到以下的輸出內容：

```
Created secret: projects/<PROJECT_NUM>/secrets/my_secret_value
```

注意：如果收到 `PermissionDenied` 錯誤 (403)，請返回「啟用 Secret Manager API」步驟。

新增密鑰版本

密鑰建立完成後，您就可以建立版本，為密鑰指派 *值*。

將下列程式碼複製到 IPython 工作階段：

```
def add_secret_version(secret_id, payload):
    # Create the Secret Manager client.
    client = secretmanager.SecretManagerServiceClient()

    # Build the resource name of the parent secret.
    parent = f"projects/{PROJECT_ID}/secrets/{secret_id}"

    # Convert the string payload into a bytes. This step can be omitted if you
    # pass in bytes instead of a str for the payload argument.
    payload = payload.encode('UTF-8')

    # Add the secret version.
    response = client.add_secret_version(parent=parent, payload={'data': payload})

    # Print the new secret version name.
    print(f'Added secret version: {response.name}')
```

呼叫函式來建立及新增密鑰版本：

```
add_secret_version("my_secret_value", "Hello Secret Manager")
```

您應該會看到以下的輸出內容：

```
Added secret version: projects/<PROJECT_NUM>/secrets/my_secret_value/versions/1
```

密鑰可以有多個版本。使用不同值再次呼叫函式：

```
add_secret_version("my_secret_value", "Hello Again, Secret Manager")
```

您應該會看到以下的輸出內容：

```
Added secret version: projects/<PROJECT_NUM>/secrets/my_secret_value/versions/2
```

請注意，新版密鑰比原始密鑰長得多。稍後會參照這個屬性。

7. 存取密鑰

存取密鑰版本會傳回密鑰內容，以及密鑰版本的其他中繼資料。存取密鑰版本時，您可以指定特定版本，或指定「latest」來要求最新版本。

警告：請勿在正式版中列印密碼值。

密鑰應嚴格保密。將資料庫憑證儲存為密鑰，然後用於驗證，或儲存認證並使用認證；但請勿直接列印密鑰，因為這樣就失去密鑰的保密意義。

您將對密鑰執行作業，評估其價值，但不會直接列印出來。您會改為輸出密碼值的「雜湊」。

將下列程式碼複製到 IPython 工作階段：

```
def access_secret_version(secret_id, version_id="latest"):
    # Create the Secret Manager client.
    client = secretmanager.SecretManagerServiceClient()

    # Build the resource name of the secret version.
    name = f"projects/{PROJECT_ID}/secrets/{secret_id}/versions/{version_id}"

    # Access the secret version.
    response = client.access_secret_version(name=name)

    # Return the decoded payload.
    return response.payload.data.decode('UTF-8')

import hashlib

def secret_hash(secret_value):
    # return the sha224 hash of the secret value
    return hashlib.sha224(bytes(secret_value, "utf-8")).hexdigest()
```

呼叫函式，以擷取密鑰的值雜湊：

```
secret_hash(access_secret_version("my_secret_value"))
```

畫面會顯示類似雜湊的輸出內容（確切值可能與這個輸出內容不符）：

```
83f8a4edb555cde4271029354395c9f4b7d79706ffa90c746e021d11
```

由於您未指定版本，系統擷取的是最新值。

呼叫函式並新增預期版本號碼，確認是否相符：

```
secret_hash(access_secret_version("my_secret_value", version_id=2))
```

您應該會看到與上一個指令相同的輸出內容：

```
83f8a4edb555cde4271029354395c9f4b7d79706ffa90c746e021d11
```

再次呼叫函式，但這次要指定第一個版本：

```
secret_hash(access_secret_version("my_secret_value", version_id=1))
```

這次您應該會看到不同的雜湊值，表示輸出內容不同：

```
9a3fc8b809ddc611c82aee950c636c7557e220893560ec2c1eeeb177
```

在程式碼中參照最新版本的密鑰，即可實作[密鑰輪替](#)。

步驟 8 · 約 3 分鐘

8. 搭配使用 Secret Manager 與 Cloud Functions

您可以在 Google Cloud 的許多部分使用密鑰。在本節中，您將著重瞭解 Google 的事件導向無伺服器運算服務 Cloud Functions。

如果您有興趣在 Cloud Functions 中使用 Python，可以按照 [Python 中的 HTTP Google Cloud Functions 程式碼實驗室](#) 操作。

呼叫 `exit` 函式，關閉 IPython：

```
exit
```

系統應會將您帶回 Cloud Shell：

```
yourname@cloudshell:~ (<PROJECT_ID>)$
```

您必須先啟用 API，才能使用 Cloud Functions API。您可以使用 Cloud Shell 執行下列指令，啟用 API：

```
gcloud services enable cloudfunctions.googleapis.com cloudbuild.googleapis.com
```

建立新資料夾來建構函式，並建立空白檔案來寫入內容：

```
mkdir secret-manager-api-demo  
cd secret-manager-api-demo  
touch main.py  
touch requirements.txt
```

從 Cloud Shell 右上方開啟程式碼編輯器：

 Open editor



前往 `secret-manager-api-demo` 資料夾中的 `main.py` 檔案。您會在這裡放置所有程式碼。

步驟 9 · 約 2 分鐘

9. 編寫 Cloud 函式以存取 Secret

雖然從指令列或 IPython 終端機儲存及擷取密鑰值很有用，但如果能在函式中存取這些密鑰，會更有幫助。

您可以使用先前建立的 `access_secret_version` 函式做為 Cloud Function 的基礎。

將下列程式碼複製到 `main.py` 檔案中：

main.py

```
import os

from google.cloud import secretmanager

project_id = os.environ["PROJECT_ID"]

client = secretmanager.SecretManagerServiceClient()
name = f"projects/{project_id}/secrets/my_secret_value/versions/latest"
response = client.access_secret_version(name=name)
my_secret_value = response.payload.data.decode("UTF-8")

def secret_hello(request):
    if "Again" in my_secret_value:
        return "We meet again!\n"

    return "Hello there.\n"
```

請注意，函式是在 Python 函式外部執行這項工作。這樣一來，Cloud 函式在例項化時只會存取 Secret 一次，而非每次要求都存取一次，因此可加快函式執行速度。

您必須先完成環境設定，才能部署函式。您必須設定函式依附元件。

建立名為 `requirements.txt` 的新檔案，並在其中加入 `google-cloud-secret-manager` 套件：

requirements.txt

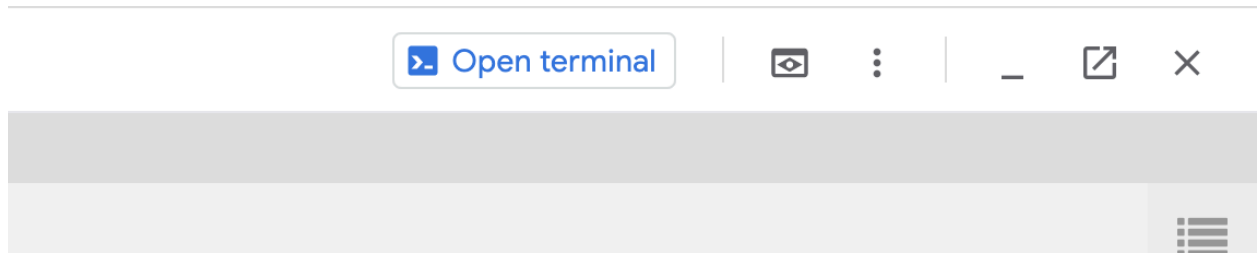
```
google-cloud-secret-manager==2.10.0
```

現在您應該會有一個資料夾，其中只包含 `main.py` 和 `requirements.txt`。

允許存取密鑰

部署函式前，請先允許 Cloud Functions 存取密鑰。

切換回終端機：



授予 Cloud Functions 服務帳戶存取密鑰的權限：

```
export PROJECT_ID=$(gcloud config get-value core/project)

gcloud secrets add-iam-policy-binding my_secret_value \
  --role roles/secretmanager.secretAccessor \
  --member serviceAccount:${PROJECT_ID}@appspot.gserviceaccount.com
```

您應該會看到以下的輸出內容：

```
Updated IAM policy for secret [my_secret_value].
bindings:
- members:
  - serviceAccount:<PROJECT_ID>@appspot.gserviceaccount.com
    role: roles/secretmanager.secretAccessor
etag: BwWiRUt2oB4=
version: 1
```


10. 部署 Cloud Function

根據先前章節的設定，您現在可以部署及測試 Cloud Function。

在只包含您建立的兩個檔案的資料夾中，部署函式：

```
gcloud functions deploy secret_hello \  
  --runtime python39 \  
  --set-env-vars PROJECT_ID=${PROJECT_ID} \  
  --trigger-http \  
  --allow-unauthenticated
```

您應該會看到下列輸出內容 (已截斷)：

```
Deploying function (may take a while - up to 2 minutes)...done.  
  
...  
  
entryPoint: secret_hello  
httpsTrigger:  
  url: https://<REGION>-<PROJECT_ID>.cloudfunctions.net/secret_hello  
...  
status: ACTIVE  
...
```

使用下列指令擷取函式的網址 (`httpsTrigger.url` 中繼資料)：

```
FUNCTION_URL=$(gcloud functions describe secret_hello --format  
'value(httpsTrigger.url)')
```

現在，請呼叫函式，測試是否能存取函式並取得預期傳回值：

```
curl $FUNCTION_URL
```

您應該會看到以下的輸出內容：

```
We meet again!
```

這個函式參照的是最新版本的密鑰，該版本已設為包含「Again」字串，因此這個函式的運作方式符合預期。

11. 恭喜！

您已學會如何使用 Python 存取 Secret Manager API！

清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取您在本教學課程中所用資源的相關費用：

- 前往 Cloud Console 中的「管理資源」頁面。
- 在專案清單中選取專案，然後按一下「刪除」。
- 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關閉) 刪除專案。

瞭解詳情

- Secret Manager：<https://cloud.google.com/secret-manager/>
- Google Cloud 上的 Python：<https://cloud.google.com/python/>
- Python 適用的 Cloud 用戶端程式庫：<https://googlecloudplatform.github.io/google-cloud-python/>

授權

這項內容採用的授權為 創用 CC 姓名標示 2.0 通用授權。
